

ЛР12 (3 пары / 6 часов)

ЛР12 (3 пары / 6 часов)

Интерфейсы и "подмена" хранения: ITaskStorage + JsonStorage + MemoryStorage

Подготовка к тестам и модульности

0) Цель лабораторной работы

Научиться делать так, чтобы проект **не зависел от конкретного способа хранения задач**.

Мы хотим уметь выбирать:

- **JSON (реальный режим)** — задачи сохраняются в файл
 - **Memory (тестовый режим)** — задачи хранятся только в памяти и пропадают после выхода
-

1) Что должно быть готово до ЛР12 (проверка базы)

Перед началом у тебя уже должно быть:

 TaskTracker.Core

- TaskItem, TaskStatus
- TaskService (Add, GetAll, Update, ChangeStatus, Delete, ReplaceAll, Search/Filter/Sort, GetStats)

 TaskTracker.Storage

- JsonTaskStorage
- BackupService

- `AppLogger`

✓ `TaskTracker.App`

- консольное меню, сохранение в JSON, логирование, импорт/экспорт/backup, статистика

✓ В проекте есть папка `data/tasks.json`

2) Итог ЛР12 (чек-лист результата)

В конце ЛР12 должно работать:

✓ В Core есть интерфейс `ITaskStorage`

✓ В Core есть `MemoryTaskStorage`

✓ В Storage есть `JsonTaskStorageAdapter`

✓ В App при запуске можно выбрать режим хранения

✓ В коде приложения больше **нет прямой привязки** к `JsonTaskStorage`

✓ Всё продолжает работать как раньше

3) Теория (развернуто, но по делу)

3.1 Что такое интерфейс простыми словами

Интерфейс — это **обещание**, что объект умеет делать нужные действия.

Например, “хранилище задач” обязано уметь:

- `Load()` — загрузить список задач
- `Save()` — сохранить список задач

Как именно оно это делает — не важно:

- JSON-файл ✓
 - память ✓
 - база данных ✓ (будет позже)
-

3.2 Что такое “подмена” (замена реализации)

У нас есть один “договор”:

| `ITaskStorage`

Но можно сделать разные реализации:

- `JsonTaskStorageAdapter` → работает с файлом
- `MemoryTaskStorage` → работает в памяти

TaskService / App не должны знать, какое именно хранилище внутри.

3.3 Почему это важно для тестов

Тесты должны быть:

- быстрые
- независимые от компьютера
- без файлов

Поэтому в тестах будет выгодно использовать **MemoryTaskStorage**.

4) Практика (Пара 1) — интерфейс `ITaskStorage` в Core

✓ Задача пары:

Создать интерфейс “хранилище задач”.

Шаг 1. Открыть решение

1. Visual Studio → открыть `TaskTracker.sln`
2. **Build** → **Build Solution**

✓ должно быть *Build succeeded*

Шаг 2. Создать папку Storage

TaskTracker.Core → правой кнопкой → Add → New Folder

Название: **Storage**

Шаг 3. Создать файл ITaskStorage.cs

TaskTracker.Core → Storage → Add → Class...

Название: **ITaskStorage.cs**

 Удали всё и вставь:

```
using TaskTracker.Core.Models;

namespace TaskTracker.Core.Storage;

public interface ITaskStorage
{
    List<TaskItem> Load();
    void Save(List<TaskItem> tasks);
}
```

Шаг 4. Проверить сборку

Build → Build Solution

 *Build succeeded*

Коммит №1 (через GitHub Desktop)

GitHub Desktop:

1. Открыть GitHub Desktop
 2. Слева выбрать репозиторий TaskTracker
 3. Вкладка **Changes**
 4. В Summary написать:
-

feat: add ITaskStorage interface in Core

1. Нажать **Commit to main**
2. Нажать **Push origin**

✓ ГОТОВО

5) Практика (Пара 2) — MemoryTaskStorage в Core

✓ Задача пары:

Сделать хранилище в памяти (для тестов).

Шаг 5. Создать MemoryTaskStorage.cs

TaskTracker.Core → Storage → Add → Class...

Название: `MemoryTaskStorage.cs`

Вставь:

```
using System.Linq;
using TaskTracker.Core.Models;

namespace TaskTracker.Core.Storage;

public class MemoryTaskStorage : ITaskStorage
{
    private List<TaskItem> _tasks = new();

    public List<TaskItem> Load()
    {
        // Возвращаем копию
        return _tasks.Select(t => new TaskItem
        {
            Id = t.Id,
            Title = t.Title,
```

```

        Description = t.Description,
        Status = t.Status
    }).ToList();
}

public void Save(List<TaskItem> tasks)
{
    // Сохраняем копию (защита от случайных изменений извне)
    _tasks = tasks.Select(t => new TaskItem
    {
        Id = t.Id,
        Title = t.Title,
        Description = t.Description,
        Status = t.Status
    }).ToList();
}
}

```

Шаг 6. Сборка

Build → Build Solution

✓ Build succeeded

✓ КОММИТ №2 (GitHub Desktop)

Summary:

feat: add MemoryTaskStorage implementation

Commit → Push

6) Практика (Пара 3) — JsonTaskStorageAdapter + выбор режима

в App

✓ Задача пары:

Сделать адаптер для JSON, который подходит под интерфейс, и обновить Program.cs.

Шаг 7. Создать JsonTaskStorageAdapter.cs

В проекте `TaskTracker.Storage → Services`:

Add → Class...

Название: `JsonTaskStorageAdapter.cs`

Вставь:

```
using TaskTracker.Core.Models;
using TaskTracker.Core.Storage;

namespace TaskTracker.Storage.Services;

public class JsonTaskStorageAdapter : ITaskStorage
{
    private readonly JsonTaskStorage _json;

    public JsonTaskStorageAdapter(string filePath)
    {
        _json = new JsonTaskStorage(filePath);
    }

    public List<TaskItem> Load() => _json.Load();

    public void Save(List<TaskItem> tasks) => _json.Save(tasks);
}
```

Шаг 8. Подключить Core.Storage в App

Открой `TaskTracker.App → Program.cs`

Добавь using:

```
using TaskTracker.Core.Storage;
```

Шаг 9. Сделать выбор режима хранения

Найди где было:

```
var storage = new JsonTaskStorage(dataFilePath);
```

ЗАМЕНИ на:

```
ITaskStorage storage;

Console.WriteLine("Выберите хранилище:");
Console.WriteLine("1 - JSON файл (обычный режим)");
Console.WriteLine("2 - Memory (тестовый режим, данные пропаду  
т после выхода)");
Console.Write("Ваш выбор: ");

var mode = (Console.ReadLine() ?? "").Trim();

if (mode == "2")
{
    storage = new MemoryTaskStorage();
    Console.WriteLine("Режим: MemoryStorage");
}
else
{
    storage = new JsonTaskStorageAdapter(dataFilePath);
}
```



```
Console.WriteLine("Режим: JsonStorage");  
}
```

⚠ Важно: убедись что App имеет ссылки на проекты:

- TaskTracker.Core
- TaskTracker.Storage

Шаг 10. Загрузка задач (как раньше)

Оставь:

```
var loadedTasks = storage.Load();  
var service = new TaskService(loadedTasks);  
Console.WriteLine($"Загружено задач: {loadedTasks.Count}");
```

Шаг 11. Сохранение — не меняем

Везде где раньше было:

```
storage.Save(service.GetAll());
```

✅ оставляем так же — потому что теперь это интерфейс

Шаг 12. Проверка (F5)

✅ Проверка А: JSON режим

1. Запустить
2. выбрать **1**
3. добавить задачу
4. выйти
5. запустить снова выбрать **1**

✓ задача осталась

✓ Проверка В: Memory режим

1. Запустить
2. выбрать 2
3. добавить задачу
4. выйти
5. запустить снова выбрать 2

✓ список пуст

✓ Коммит №3 (GitHub Desktop)

Summary:

feat: add json adapter and allow selecting storage mode

Commit → Push

7) Отчёт ЛР12 (обязателен)

Создай файл:

docs/LR12_Report.md

Вставь:

```
# Отчёт ЛР12 – интерфейсы и хранилища
```

```
## Цель
```

Добавить интерфейс хранения задач и возможность подмены хранения (JSON/Memory).

```
## Что сделано
```

- Добавлен интерфейс ITaskStorage (Load/Save)
- Реализован MemoryTaskStorage (данные в памяти)
- Реализован JsonTaskStorageAdapter (адаптер для JSON)

- Приложение запускается в двух режимах:
 - JSON режим (данные сохраняются)
 - Memory режим (данные пропадают после выхода)

Как проверить

- 1) Запустить приложение и выбрать JSON режим
- 2) Добавить задачу, выйти, запустить снова -> задача осталась
- 3) Запустить приложение и выбрать Memory режим
- 4) Добавить задачу, выйти, запустить снова -> список пуст

✓ Коммит №4 (если нужен)

Summary:

```
docs: add LR12 report
```

Commit → Push

8) Частые ошибки (и как быстро исправить)

✗ "Не видит ITaskStorage"

✓ Проверь `using TaskTracker.Core.Storage;`

✗ "JsonTaskStorageAdapter не компилируется"

✓ Убедись что есть ссылки:

- Storage → Core
- App → Core и Storage

✗ "В Memory режиме всё равно сохраняется"

✓ Значит ты где-то создаёшь `JsonTaskStorage` напрямую.

Ищи по проекту:

`new JsonTaskStorage(`

и убирай.

9) Что сдавать (чек-лист)

✓ GitHub ссылка

✓ `docs/LR12_Report.md`

✓ Видео/скрин 30–60 сек:

- запуск → выбор JSON → добавил → перезапуск → задача осталась
- запуск → выбор Memory → добавил → перезапуск → пусто

✓ минимум 3 коммита (лучше 4)

10) Оценивание

3 — есть ITaskStorage + JSON работает через адаптер

4 — + Memory режим

5 — аккуратная структура + нет прямых зависимостей от JsonTaskStorage в App
